

Project Report: Digital Screen Time Analyzer

A Streamlined, Modular Terminal-Based Screen Time Tracking Solution

Prepared By: Swayam Gupta (GitHub: swayamgupta)

Course: CSE (AI and ML)

Submission Portal: VITYARTHI Portal

Date: September 2026

1. Cover Page

The cover page includes the Vityarthi Learning Platform header, project title, subtitle, author details (Swayam Gupta, GitHub ID: swayamgupta), course name (CSE (AI and ML)), submission portal, and date [cite: 1].

2. Introduction

The Digital Screen Time Analyzer is a terminal-based utility designed to help students track their daily digital activities [cite: 2]. By logging app usage durations and categorizing tasks into productive and recreational domains, users can monitor their screen time habits without distraction or complex graphical interfaces [cite: 2].

3. Problem Statement

Students frequently lose track of time spent on unproductive digital platforms (such as gaming and social media) versus productive commitments like coding and studying. Existing solutions are often cloud-dependent, bloated, or require heavy GUI configurations. There is a need for a lightweight, secure, command-line tool for local digital habit monitoring [cite: 2].

4. Functional Requirements

- **Activity Logging:** Users can input application names, select categories (Productive, Education, Entertainment, Social Media, Gaming), and specify durations in minutes [cite: 2].
- **Data Reporting:** The system reads stored logs and calculates overall metrics to display a comprehensive report card [cite: 2].
- **Visual Charts:** The application generates text-based bar charts inside the terminal to visualize time distribution across categories [cite: 2].
- **Data Management:** Users can reset or clear all stored records safely via confirmation prompts.

5. Non-functional Requirements

- **Privacy & Security:** Zero external server dependency; all records reside in a local CSV file [cite: 2].
- **Performance:** Instant command execution without terminal lag or blocking loops.
- **Usability:** Clean, interactive numbered menu system (Options 1–5) with robust input validation [cite: 2].
- **Portability:** Runs natively on any system with Python 3 installed, requiring zero external `pip` packages.

6. System Architecture

The application utilizes a modular design pattern separating responsibilities across dedicated Python scripts:

- **Presentation Layer:** Command-line interface (`main.py`) handling user inputs and menu navigation [cite: 2].
- **Business Logic Layer:** Object-oriented session wrapping (`add\_data.py`) and analytics computation (`show\_graphs.py`) [cite: 2].
- **Persistence Layer:** Flat-file CSV storage layer (`file\_read\_write.py`) [cite: 2].

7. Design Diagrams

- **Use Case Diagram:** User interacts with the CLI menu to log usage, view reports, render charts, or reset data.
- **Workflow Diagram:** Program loop -> Menu Display -> User Choice -> Module Delegation -> CSV Read/Write -> Terminal Output.
- **Sequence Diagram:** ``main.py`` passes input to ``add_data.ScreenSession`` -> ``file_read_write.save_new_record()`` appends row to CSV.
- **Class/Component Diagram:** ``ScreenSession`` class interacting with ``file_read_write`` and ``show_graphs`` utility functions [cite: 2].
- **ER Diagram / Schema:** Single relational table stored in CSV format with columns: ``App``, ``Category``, ``Minutes``, ``Timestamp``.

8. Design Decisions & Rationale

- Choice of CSV Storage: Chosen over SQL databases to keep the project completely dependency-free and lightweight.
- Modular Script Separation: Divided into 4 distinct files (``main.py``, ``add_data.py``, ``file_read_write.py``, ``show_graphs.py``) to enhance maintainability and adhere to clean code principles [cite: 2].

9. Implementation Details

Core code snippets demonstrating modular integration:

```
# main.py loop snippet
user_choice = input('Enter your choice (1-5): ')
if user_choice == '1':
    s = add_data.ScreenSession(app, cat, mins, now)
    file_read_write.save_new_record(s.make_list())
```

10. Screenshots / Results

Terminal execution preview:

```
=== SCREEN TIME TRACKER MENU ===
1. Log new app usage
2. Show my report card
3. Show bar chart graph
4. Reset all data
5. Exit program
```

11. Testing Approach

- Unit verification of inputs (ensuring numeric validation on minute fields) [cite: 2].
- Integration testing between ``file_read_write.py`` and CSV data persistence.
- Boundary testing with empty dataset handling and reset confirmations.

12. Challenges Faced

- Handling invalid string inputs when integer minutes were expected.
- Managing clean ASCII formatting for terminal-based bar charts across different console window sizes.

13. Learnings & Key Takeaways

- Gained practical experience in structuring multi-file Python applications.
- Mastered flat-file data persistence using Python's built-in ``csv`` module [cite: 2].
- Strengthened command-line user interface design principles.

14. Future Enhancements

- Adding weekly and monthly trend analysis reports.
- Implementing export features for PDF/HTML executive summaries.
- Developing a lightweight desktop GUI wrapper using Tkinter.

15. References

- Python Standard Library Documentation (`csv`, `datetime`, `os`).
- Vityarathi Learning Platform Python Essentials Curriculum Guidelines.